

This means that the success of the CSRF exploit depends on the browser sending the session cookies along with the http request. Thus, the obvious protection mechanism is to direct the browser in which conditions the session cookies are to be sent with the request.

SameSite cookie attributes

The browsers follow SOP (Same-Origin Policy) when domain cookies are sent for the specific request. The introduction of SameSite is an extension of this ability, where the server directs the browser in which category of requests the latter should include the session cookies. You can refer to <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie/SameSite> for the browsers that support the SameSite attribute.

There are three possible values of the SameSite attribute:

- None
- Lax
- Strict

This attribute is used as one of the protections against CSRF attacks.

CORS (cross-origin resource sharing)

Applications are expected to accept cross-site requests, i.e., requests coming from other domains. With strict SOP, browsers will not make any cross-

domain requests, which may hamper the functioning of these applications. CORS is the answer to this challenge. Here, the application server indicates which domains are allowed to make requests to the application, which methods can be used in the endpoint while making such requests, and which requests the browser should send with domain cookies.

Since the server relies on browser support for CORS, it becomes necessary to understand how browsers decide which cross-domain requests are to be sent to the server and, if so, whether the domain cookies are to be sent along with the request. Let us understand browser handling for this.

Modern browser implementations treat requests in two categories: simple and non-simple (pre-flight).

To identify the request category, the browser uses the following parameters:

- Request verb
- Content/media type of the request
- Type of the request as cross-site/same-origin/same-site

Criteria for simple requests

For the following parameters, browsers treat the request to be a simple request and send it as such to the server without any pre-flight.

Simple request safest headers and the values

Methods: POST, GET, HEAD

Safe listed request headers: Accept, Accept-Language, Content-Language, Content-Type

Media types: text/plain, multipart/form-data, application/x-www-form-urlencoded

GET and HEAD methods do not need media type. Browsers treat such requests as simple requests. For POST requests, the media types mentioned above are that of the simple request. For any other media types other than the ones mentioned above for POST request, the request is non-simple and the browser makes a pre-flight before sending the actual request.

What does pre-flight mean?

Once the browser analyses and identifies specific requests to be non-simple — for example, XHR POST request with content-type as 'application/json' — it does not send this POST request directly to the server. It first checks if the server is ready to accept such a request or not, and if yes, in what conditions it will do so. This check is done with the help of the OPTIONS method, called a 'pre-flight' request, where the browser adds headers in the request to get clarity on various parameters allowed by the server.

The CORS request headers are as follows:

- Access-Control-Request-Headers
- Access-Control-Request-Method

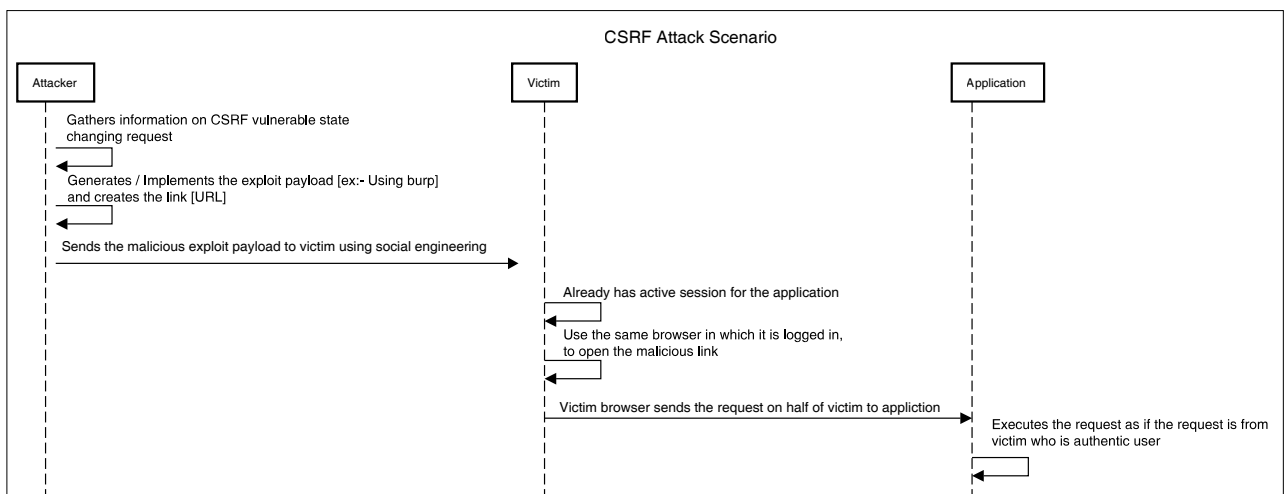


Figure 1: CSRF attack